

Tema 10 — Introducción a la Biblioteca estándar del lenguaje

Laboratorio Guiado

Objetivos	Practicar el uso de módulos de la biblioteca estándar de Python en tareas de soporte TI: sistema y entorno, rutas, ficheros, temporales, procesos externos, formatos de datos, logging, fechas, seguridad básica, SQLite, HTTP y un ejercicio integrado.
Material necesario	Terminal, Python 3.13, venv del Tema10, Visual Studio Code con extensiones Python y Jupyter, scripts del tema instalados en ~/Curso_Python/Tema10/src.

Laboratorio 1. Sistema y entorno con os, sys y platform

1. Actualizar el repositorio, abrir la carpeta del tema y activar el entorno virtual. El venv ya está creado.

```
git pull origin main
cd ~/Curso_Python/Tema10/src
source ../.venv/bin/activate
python --version
code ..
```

2. Abrir el fichero 01_sistema_entorno.py. Revisar la importación de os, sys y platform.

```
import os
import sys
import platform
```

3. Obtener información del intérprete Python. sys permite consultar la versión y la ruta del ejecutable activo.

```
print("=== Información del intérprete Python ===")
print("Versión de Python:", sys.version.split()[0])
print("Ejecutable Python:", sys.executable)
```

4. Obtener información del proceso y del usuario. os permite acceder al entorno y al directorio de ejecución.

```
print("\n=== Información del proceso y usuario ===")
print("Usuario:", os.environ.get("USER") or os.environ.get("USERNAME") or
      "desconocido")
print("PID actual:", os.getpid())
print("Directorio actual:", os.getcwd())
```

5. Consultar información del sistema operativo con platform.

```
print("\n=== Información del sistema operativo ===")
print("Sistema:", platform.system())
print("Release:", platform.release())
print("Versión:", platform.version())
```

```
print("Máquina:", platform.machine())
```

6. Ejecutar el script completo desde la terminal.

```
python 01_sistema_entorno.py
```

Resultado esperado: el alumno debe obtener contexto básico de Python, proceso, usuario, directorio y sistema operativo sin instalar paquetes externos.

Laboratorio 2. Rutas, ficheros, copias y búsqueda por patrón

1. Abrir el fichero 02_rutas_ficheros.py. Revisar los módulos pathlib, shutil y glob.

```
from pathlib import Path
import shutil
import glob

BASE_DIR = Path(__file__).resolve().parent.parent
DATA_DIR = BASE_DIR / "data"
BACKUP_DIR = BASE_DIR / "backup"
```

2. Preparar los directorios data y backup. pathlib permite crear rutas de forma portable.

```
print("=== 1. Preparar directorios ===")
DATA_DIR.mkdir(parents=True, exist_ok=True)
BACKUP_DIR.mkdir(parents=True, exist_ok=True)
print("Data :", DATA_DIR)
print("Backup:", BACKUP_DIR)
```

3. Crear ficheros de ejemplo en el directorio data usando write_text().

```
print("\n=== 2. Crear ficheros JSON de ejemplo ===")
(DATA_DIR / "config1.json").write_text('{"servicio": "ssh", "estado": "activo"}', encoding="utf-8")
(DATA_DIR / "config2.json").write_text('{"servicio": "nginx", "estado": "activo"}', encoding="utf-8")
(DATA_DIR / "notas.txt").write_text("Fichero auxiliar", encoding="utf-8")
print("Ficheros creados en:", DATA_DIR)
```

4. Copiar el directorio completo con shutil.copytree(). dirs_exist_ok=True permite repetir el laboratorio sin error.

```
print("\n=== 3. Copiar el directorio data a backup/data_copia ===")
destino_copia = BACKUP_DIR / "data_copia"
shutil.copytree(DATA_DIR, destino_copia, dirs_exist_ok=True)
print("Copia creada en:", destino_copia)
```

5. Localizar ficheros JSON con glob usando un patrón.

```
print("\n=== 4. Buscar ficheros JSON con glob ===")
patron = str(destino_copia / "*.json")
archivos = glob.glob(patron)

for archivo in archivos:
```

```
print("JSON respaldado:", archivo)
```

6. Ejecutar el script completo desde la terminal.

```
python 02_rutas_ficheros.py
```

Resultado esperado: el alumno debe crear rutas de trabajo, generar ficheros, copiarlos a backup y localizar ficheros por extensión.

Laboratorio 3. Ficheros y directorios temporales con tempfile

1. Abrir el fichero 03_temporales.py. Revisar el uso de tempfile y pathlib.

```
import tempfile
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
DATA_DIR = BASE_DIR / "data"
DATA_DIR.mkdir(parents=True, exist_ok=True)
```

2. Crear un directorio temporal dentro de data usando TemporaryDirectory().

```
print("=== Crear directorio temporal controlado ===")
with tempfile.TemporaryDirectory(dir=DATA_DIR) as tmp_dir:
    directorio = Path(tmp_dir)
    archivo = directorio / "dump_db.sql"
```

3. Escribir un archivo temporal y comprobar que existe mientras el contexto sigue abierto.

```
print("Temporal creado:", directorio)
archivo.write_text("SELECT * FROM usuarios;
", encoding="utf-8")

print("Archivo temporal:", archivo)
print("Existe dentro del contexto:", archivo.exists())
print("Contenido:", archivo.read_text(encoding="utf-8").strip())
```

4. Comprobar que el directorio temporal se elimina automáticamente al salir del bloque with.

```
print("\n=== Después del contexto ===")
print("El directorio temporal sigue existiendo:", directorio.exists())
```

5. Ejecutar el script completo desde la terminal.

```
python 03_temporales.py
```

Resultado esperado: el alumno debe comprender cómo crear recursos temporales seguros y cómo Python los limpia automáticamente al salir del contexto.

Laboratorio 4. Procesos externos con subprocess

1. Abrir el fichero 04_procesos_subprocess.py. Revisar la función auxiliar ejecutar_comando().

```
import subprocess

def ejecutar_comando(comando):
    """Ejecuta un comando externo y muestra salida controlada."""
    print("Comando:", " ".join(comando))
```

2. Ejecutar el comando con subprocess.run() usando lista de argumentos, no una cadena construida manualmente.

```
resultado = subprocess.run(
    comando,
    capture_output=True,
    text=True,
    check=False,
)

print("Código de retorno:", resultado.returncode)
```

3. Mostrar stdout y stderr si existen. El código de retorno permite saber si el comando terminó correctamente.

```
if resultado.stdout:
    print("--- stdout ---")
    print(resultado.stdout.strip())

if resultado.stderr:
    print("--- stderr ---")
    print(resultado.stderr.strip())

return resultado.returncode
```

4. Probar un comando real para comprobar el espacio en disco.

```
print("=== 1. Comprobar espacio en disco ===")
ejecutar_comando(["df", "-h", "/"])
```

5. Probar un comando con código de retorno distinto de cero para observar un fallo controlado.

```
print("\n=== 2. Comando con código de retorno distinto de cero ===")
# El comando false existe en sistemas Unix/Linux y devuelve código 1.
# Sirve para demostrar un fallo controlado sin depender de un ejecutable inexistente.
ejecutar_comando(["false"])
```

6. Ejecutar el script completo desde la terminal.

```
python 04_procesos_subprocess.py
```

Resultado esperado: el alumno debe ejecutar comandos externos de forma controlada, capturar salidas y revisar códigos de retorno.

Laboratorio 5. Formatos de datos con csv, json, configparser y tomlib

1. Abrir el fichero 05_formatos_datos.py. Revisar los módulos estándar usados para formatos de intercambio y configuración.

```
import csv
import json
import configparser
import tomlib
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
DATA_DIR = BASE_DIR / "data"
DATA_DIR.mkdir(parents=True, exist_ok=True)
```

2. Crear y leer un CSV usando csv.DictWriter y csv.DictReader.

```
print("=== 1. CSV: escribir y leer datos tabulares ===")
ruta_csv = DATA_DIR / "servicios.csv"
filas = [
    {"servicio": "ssh", "puerto": 22, "estado": "OK"},
    {"servicio": "nginx", "puerto": 443, "estado": "OK"},
    {"servicio": "api", "puerto": 8080, "estado": "ERROR"},
]

with open(ruta_csv, "w", newline="", encoding="utf-8") as fichero:
    escritor = csv.DictWriter(fichero, fieldnames=["servicio", "puerto", "estado"])
    escritor.writeheader()
    escritor.writerows(filas)

with open(ruta_csv, "r", newline="", encoding="utf-8") as fichero:
    lector = csv.DictReader(fichero)
    for fila in lector:
        print(f"{fila['servicio']} -> {fila['puerto']} [{fila['estado']}]")
```

3. Crear y leer JSON usando json.dumps() y json.loads().

```
print("\n=== 2. JSON: escribir y leer datos estructurados ===")
ruta_json = DATA_DIR / "servicios.json"
ruta_json.write_text(json.dumps(filas, indent=4, ensure_ascii=False), encoding="utf-8")

servicios = json.loads(ruta_json.read_text(encoding="utf-8"))
print("Servicios cargados desde JSON:", len(servicios))
```

4. Crear y leer un fichero INI con configparser.

```
print("\n=== 3. INI con configparser ===")
ruta_ini = DATA_DIR / "database.ini"
ruta_ini.write_text(
    "[mysqld]
    host = localhost
    port = 3306
    user = soporte
"
```

```
",
    encoding="utf-8",
)

config_ini = configparser.ConfigParser()
config_ini.read(ruta_ini, encoding="utf-8")
print("Puerto extraído de INI:", config_ini["mysqld"]["port"])
```

5. Crear y leer un fichero TOML con tomlib. tomlib lee en modo binario.

```
print("\n=== 4. TOML con tomlib ===")
ruta_toml = DATA_DIR / "app_config.toml"
ruta_toml.write_text(
    "[server]
    "
    "host = '127.0.0.1'
    "
    "port = 8000
    "
    "debug_mode = true
    ",
    encoding="utf-8",
)

with open(ruta_toml, "rb") as fichero:
    config_toml = tomlib.load(fichero)

print("Modo debug de TOML:", config_toml["server"]["debug_mode"])
```

6. Ejecutar el script completo desde la terminal.

```
python 05_formatos_datos.py
```

Resultado esperado: el alumno debe crear y leer CSV, JSON, INI y TOML sin usar paquetes externos.

Laboratorio 6. Registros operativos con logging

1. Abrir el fichero 06_logging.py. Preparar el directorio logs y la ruta del fichero de log.

```
from pathlib import Path
import logging

BASE_DIR = Path(__file__).resolve().parent.parent
LOG_DIR = BASE_DIR / "logs"
LOG_DIR.mkdir(parents=True, exist_ok=True)
LOG_FILE = LOG_DIR / "soporte.log"
```

2. Configurar logging para escribir eventos técnicos en un fichero.

```
# force=True permite reconfigurar logging si el script se relanza desde un entorno interactivo.
logging.basicConfig(
    filename=LOG_FILE,
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
    force=True,
)
```

3. Registrar eventos informativos y avisos. logging reemplaza a print() para trazas operativas.

```
print("=== Registrar operación ===")
logging.info("Inicio de comprobación de infraestructura.")
logging.warning("Latencia alta detectada en el servicio API.")
```

4. Registrar un error técnico y mostrar al usuario un mensaje controlado.

```
try:
    raise ConnectionRefusedError("Conexión rechazada por el puerto 5432")
except ConnectionRefusedError as error:
    logging.error("Fallo en nodo srv-db-01: %s", error)
    print("Error: La base de datos no está disponible en este momento.")
```

5. Leer las últimas líneas del log generado.

```
print("Log generado en:", LOG_FILE)
print("Últimas líneas del log:")
print(LOG_FILE.read_text(encoding="utf-8").strip().splitlines()[-3:])
```

6. Ejecutar el script completo desde la terminal.

```
python 06_logging.py
```

Resultado esperado: el alumno debe diferenciar mensajes de usuario y registros técnicos persistentes en un fichero log.

Laboratorio 7. Fechas, horas y zonas con datetime, zoneinfo y calendar

1. Abrir el fichero 07_fechas_zonas.py. Revisar la importación de datetime, zoneinfo y calendar.

```
from datetime import datetime, timezone
from zoneinfo import ZoneInfo
import calendar
```

2. Generar una marca de tiempo en UTC. UTC es apropiado para registros técnicos y correlación de eventos.

```
print("=== Fecha y hora actual en UTC ===")
ahora_utc = datetime.now(timezone.utc)
print("UTC:", ahora_utc.isoformat(timespec="seconds"))
```

3. Calcular el último día del mes con calendar.monthrange().

```
print("\n=== Calcular último día del mes ===")
anio = ahora_utc.year
mes = ahora_utc.month
_, ultimo_dia = calendar.monthrange(anio, mes)
print("Año:", anio)
print("Mes:", mes)
print("Último día:", ultimo_dia)
```

4. Crear una ventana de mantenimiento en zona local usando zoneinfo.

```
print("\n=== Ventana de mantenimiento en zona local ===")
zona_local = ZoneInfo("Europe/Madrid")
ejecucion_local = datetime(anio, mes, ultimo_dia, 22, 0, tzinfo=zona_local)
print("Planificación local:", ejecucion_local.isoformat(timespec="seconds"))
```

5. Ejecutar el script completo desde la terminal.

```
python 07_fechas_zonas.py
```

Resultado esperado: el alumno debe generar marcas temporales, calcular fechas de calendario y manejar una zona horaria explícita.

Laboratorio 8. Seguridad básica con hashlib, hmac, secrets y random

1. Abrir el fichero 08_seguridad_basica.py. Revisar módulos de seguridad y aleatoriedad.

```
import hashlib
import hmac
import secrets
import random
```

2. Generar un token seguro con secrets. Este módulo es adecuado para valores con implicaciones de seguridad.

```
print("=== 1. Token temporal seguro con secrets ===")
token = secrets.token_urlsafe(12)
print("Token temporal seguro:", token)
```

3. Calcular una huella SHA-256 del token con hashlib.

```
print("\n=== 2. Huella SHA-256 con hashlib ===")
huella = hashlib.sha256(token.encode("utf-8")).hexdigest()
print("SHA-256:", huella)
```

4. Crear una firma HMAC para un mensaje. HMAC combina una clave y un mensaje.

```
print("\n=== 3. Firma HMAC de un mensaje ===")
clave = secrets.token_bytes(16)
mensaje = b"operacion=backup;host=srv-db-01"
firma = hmac.new(clave, mensaje, hashlib.sha256).hexdigest()
print("Firma HMAC:", firma)
```

5. Usar random solo para selecciones no críticas, como pruebas o simulaciones.

```
print("\n=== 4. random para uso no crítico ===")
servidor = random.choice(["srv-web-01", "srv-web-02", "srv-web-03"])
print("Servidor elegido para prueba no crítica:", servidor)
```

6. Ejecutar el script completo desde la terminal.

```
python 08_seguridad_basica.py
```


Resultado esperado: el alumno debe distinguir generación segura con secrets, huellas, HMAC y uso no crítico de random.

Laboratorio 9. Persistencia ligera con sqlite3

1. Abrir el fichero 09_sqlite3_persistencia.py. Preparar una base SQLite local en data.

```
import sqlite3
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
DATA_DIR = BASE_DIR / "data"
DATA_DIR.mkdir(parents=True, exist_ok=True)
DB_FILE = DATA_DIR / "inventario.db"
```

2. Crear o abrir la base de datos y definir una tabla si no existe.

```
print("=== Crear o abrir base SQLite ===")
bd = sqlite3.connect(DB_FILE)

bd.execute("""
CREATE TABLE IF NOT EXISTS nodos (
    host TEXT,
    ip TEXT,
    rol TEXT
)
""")
```

3. Insertar registros de forma reproducible. DELETE permite repetir el laboratorio sin duplicados.

```
# Limpiamos la tabla para que el script sea reproducible.
bd.execute("DELETE FROM nodos")

datos = [
    ("web-01", "10.0.1.10", "frontend"),
    ("db-01", "10.0.1.20", "backend"),
    ("cache-01", "10.0.1.30", "redis"),
]

bd.executemany("INSERT INTO nodos VALUES (?, ?, ?)", datos)
bd.commit()

print("Registros insertados:", len(datos))
```

4. Consultar registros con SQL parametrizado.

```
print("\n=== Consultar nodos backend ===")
for fila in bd.execute("SELECT host, ip FROM nodos WHERE rol = ?", ("backend",)):
    print(f"Servidor crítico: {fila[0]} localizado en {fila[1]}")

bd.close()
print("Base de datos:", DB_FILE)
```

5. Ejecutar el script completo desde la terminal.

```
python 09_sqlite3_persistencia.py
```

Resultado esperado: el alumno debe crear una base SQLite local, insertar datos y consultar registros con SQL básico.

Laboratorio 10. HTTP básico con urllib

1. Abrir el fichero 10_http_urllib.py. Revisar urllib.request y urllib.error.

```
from urllib.request import urlopen
from urllib.error import URLError
import json

url = "https://api.ipify.org?format=json"
```

2. Realizar una petición HTTP con timeout y leer la respuesta JSON.

```
print("=== Petición HTTP básica ===")
print("URL:", url)

try:
    with urlopen(url, timeout=5) as respuesta:
        datos = json.load(respuesta)
        print("Conexión exitosa.")
        print("IP pública detectada:", datos["ip"])
```

3. Capturar errores de conexión o timeout. Este laboratorio puede fallar si no hay salida a Internet.

```
except URLError as error:
    print("No se pudo conectar con la API.")
    print("Motivo:", error.reason)
except TimeoutError:
    print("La petición superó el tiempo de espera.")
```

4. Ejecutar el script completo desde la terminal.

```
python 10_http_urllib.py
```

Resultado esperado: el alumno debe realizar una petición HTTP simple y controlar fallos de conectividad sin detener el programa abruptamente.

Laboratorio 11. Ejemplo práctico integrado de soporte

1. Abrir el fichero 11_ejemplo_integrado_soporte.py. Revisar los módulos que se combinan en el proceso.

```
import os
import sys
import json
import shutil
import hashlib
import logging
import platform
from pathlib import Path
from datetime import datetime, timezone
from collections import Counter
```

2. Preparar directorios de datos, salida, backup y logs. Este patrón organiza los resultados del script.

```
BASE_DIR = Path(__file__).resolve().parent.parent
DATA_DIR = BASE_DIR / "data"
OUTPUT_DIR = BASE_DIR / "output"
BACKUP_DIR = BASE_DIR / "backup"
LOG_DIR = BASE_DIR / "logs"

for directorio in (DATA_DIR, OUTPUT_DIR, BACKUP_DIR, LOG_DIR):
    directorio.mkdir(parents=True, exist_ok=True)
```

3. Configurar logging para registrar la operación integrada.

```
LOG_FILE = LOG_DIR / "operacion_integrada.log"

logging.basicConfig(
    filename=LOG_FILE,
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
    force=True,
)
```

4. Definir una función para obtener contexto del entorno de ejecución.

```
def obtener_contexto():
    """Devuelve información básica del entorno de ejecución."""
    return {
        "python": sys.version.split()[0],
        "ejecutable": sys.executable,
        "sistema": platform.system(),
        "release": platform.release(),
        "maquina": platform.machine(),
        "usuario": os.environ.get("USER") or os.environ.get("USERNAME") or
        "desconocido",
        "directorio_actual": os.getcwd(),
        "timestamp_utc": datetime.now(timezone.utc).isoformat(timespec="seconds"),
    }
```

5. Definir funciones de validación, normalización y cálculo de huella del reporte.

```
def puerto_valido(puerto):
    """Valida un puerto TCP/UDP."""
    return 1 <= puerto <= 65535

def normalizar_servicios(servicios):
    """Normaliza registros de servicio y añade estado de validación."""
    normalizados = []

    for servicio in servicios:
        nombre = servicio["nombre"].strip().lower()
        puerto = servicio["puerto"]
        activo = servicio["activo"]

        if activo and puerto_valido(puerto):
            estado = "OK"
        elif not puerto_valido(puerto):
            estado = "PUERTO_INVALIDO"
        else:
            estado = "DETENIDO"

        normalizados.append({
            "nombre": nombre,
            "puerto": puerto,
            "activo": activo,
            "estado": estado
        })
```

```

        normalizados.append({
            "nombre": nombre,
            "host": servicio["host"],
            "puerto": puerto,
            "activo": activo,
            "estado": estado,
        })

    return normalizados

def calcular_huella(ruta):
    """Calcula SHA-256 del fichero generado."""
    contenido = ruta.read_bytes()
    return hashlib.sha256(contenido).hexdigest()

```

6. Ejecutar la función principal: normalizar servicios, generar JSON, crear backup, calcular huella y registrar logs.

```

def main():
    """Ejecuta la operación integrada."""
    logging.info("Inicio de operación integrada.")

    servicios = [
        {"nombre": " SSH ", "host": "srv-web-01", "puerto": 22, "activo": True},
        {"nombre": "NGINX", "host": "srv-web-01", "puerto": 443, "activo": True},
        {"nombre": "api", "host": "srv-app-01", "puerto": 8080, "activo": False},
        {"nombre": "backup", "host": "srv-bkp-01", "puerto": 70000, "activo": True},
    ]

    contexto = obtener_contexto()
    registros = normalizar_servicios(servicios)
    resumen = Counter(registro["estado"] for registro in registros)

    salida = {
        "contexto": contexto,
        "total_servicios": len(registros),
        "resumen_estados": dict(resumen),
        "servicios": registros,
    }

    ruta_json = OUTPUT_DIR / "reporte_soporte.json"
    ruta_json.write_text(json.dumps(salida, indent=4, ensure_ascii=False),
                          encoding="utf-8")

    huella = calcular_huella(ruta_json)
    copia = BACKUP_DIR / "reporte_soporte.json"
    shutil.copy2(ruta_json, copia)

    logging.info("Reporte generado: %s", ruta_json)
    logging.info("Backup generado: %s", copia)
    logging.info("SHA-256 del reporte: %s", huella)
    logging.info("Fin de operación integrada.")

    print("=== Reporte integrado generado ===")
    print("JSON      :", ruta_json)
    print("Backup    :", copia)
    print("Log       :", LOG_FILE)
    print("SHA-256   :", huella)

```

```
print("Resumen :", dict(resumen))

if __name__ == "__main__":
    main()
```

7. Ejecutar el script completo desde la terminal.

```
python 11_ejemplo_integrado_soporte.py
```

Resultado esperado: el alumno debe integrar varios módulos estándar en un flujo de soporte que genera reporte JSON, log operativo, backup y huella SHA-256.

Ejercicios propuestos

1. Crear un script src/resumen_entorno.py que genere un JSON con versión de Python, sistema operativo, usuario y directorio actual.
2. Crear un script src/backup_configuraciones.py que copie todos los ficheros .json de data a backup/configs usando pathlib, shutil y glob.
3. Crear un script src/limpieza_temporal.py que use tempfile.TemporaryDirectory() para generar un fichero temporal y comprobar que se elimina al salir del contexto.
4. Crear un script src/comprobar_comandos.py que ejecute df -h / y uname -r con subprocess.run(), mostrando stdout, stderr y returncode.
5. Crear un script src/configuracion_ini_toml.py que cree una configuración INI y otra TOML, las lea y muestre varios valores.
6. Crear un script src/auditoria_logging.py que registre eventos INFO, WARNING y ERROR en logs/auditoria.log.
7. Crear un script src/token_backup.py que genere un token con secrets, calcule su SHA-256 y lo guarde en un JSON junto a una fecha UTC.
8. Crear un script src/inventario_sqlite.py que cree una tabla de servicios en SQLite, inserte varios registros y consulte solo los servicios activos.

Guía de resolución de problemas

Problema	Diagnóstico	Solución
No aparece (Tema10) en el prompt	El entorno virtual no está activado o se ha activado desde otro directorio.	Ejecutar source ../venv/bin/activate desde ~/Curso_Python/Tema10/src.
El script no encuentra las carpetas data, logs, output o backup	El script se ejecuta desde una ubicación distinta o no se han creado los directorios.	Ejecutar desde ~/Curso_Python/Tema10/src y revisar que el script usa BASE_DIR = Path(__file__).resolve().parent.parent.
FileNotFoundError al trabajar con rutas	La ruta relativa no coincide con el directorio de ejecución.	Comprobar pwd, revisar BASE_DIR y usar rutas construidas con pathlib.Path.
subprocess devuelve returncode distinto de cero	El comando externo falló, no existe o devolvió error controlado.	Revisar stdout, stderr y returncode. Evitar cadenas construidas dinámicamente con datos no validados.

El fichero log no aparece	logging no se configuró o el directorio logs no existe.	Crear LOG_DIR con <code>mkdir(parents=True, exist_ok=True)</code> y usar <code>logging.basicConfig(..., force=True)</code> .
UnicodeDecodeError al leer ficheros de texto	El fichero no está codificado en UTF-8.	Abrir con la codificación correcta o regenerar el fichero con <code>encoding="utf-8"</code> .
tomllib no permite escribir TOML	tomllib solo lee TOML en la biblioteca estándar.	Crear el fichero con <code>write_text()</code> o usar herramientas externas solo si el proyecto lo permite.
zoneinfo no encuentra la zona horaria	La zona indicada no existe o no está disponible en el sistema.	Usar nombres IANA válidos como Europe/Madrid y comprobar la instalación de datos de zona.
URLError o TimeoutError en urllib	El entorno no tiene salida a Internet, DNS no resuelve o la API no responde.	Mantener el try/except y explicar que el laboratorio debe fallar de forma controlada sin conexión.
sqlite3 genera una base con datos duplicados	El script inserta registros cada vez que se ejecuta.	Limpiar la tabla antes de insertar o usar claves y restricciones si se requiere persistencia real.
El SHA-256 cambia entre ejecuciones	El fichero contiene datos variables, como marcas de tiempo o tokens.	Verificar que el contenido sea el esperado y recalcular la huella después de generar el fichero.